ECEN4223-5080 Spring 2026
Mobile Robotics
Project 3

Due date: Submit your report by 11:59 p.m. Wednesday, April 15th, 2026 to Canvas.

I. Objectives

1. Learn how to use feedback control to enable the Rosmasterx3 to autonomously follow a desired path
2. Learn how to use SLAM in ROS 2 to build a map of the robot's environment while it is moving autonomously
3. Understand how SLAM improves localization accuracy compared to using encoder-based odometry feedback alone, even when the robot follows a simple, fixed trajectory

II. Tasks | **Submission requirements are highlighted in bold and underlined text**

1. Closed-loop robot control (holonomic Pure Pursuit Algorithm).
   In this task, you will develop a closed-loop motion control ROS 2 node that commands the Rosmasterx3 robot using a holonomic approach of the Pure Pursuit (Virtual Vehicle Method) feedback controller. The robot must follow the same square path defined in Project 2, but now using feedback from the robot's pose instead of open-loop timing.
   a. Node design
      - Path structure.
        - The robot must continuously track the square path using feedback control.
        - Motion must be smooth and continuous (no stopping at corners).
        - Since the Rosmasterx3 is an omnidirectional (mecanum) robot, sideways (lateral) motion must be used to correct path errors.
      - Lookahead-based target tracking.
        - The controller must use a lookahead point located at a fixed distance L_d ahead of the robot along the path.
        - The lookahead point must be updated continuously as the robot moves.
      - Velocity command generation.
        - Linear velocity commands must be computed based on the position of the lookahead point relative to the robot.
        - The controller must command:
          - Forward linear velocity, and

            - Sideways linear velocity.
        - Angular velocity should not be used.
- Continuous operation.
    - The robot must complete the square path continuously.
    - After finishing one full square, the robot must automatically repeat the path.
    - The robot should complete three (3) full square cycles.
- No manual intervention.
    - Keyboard or manual control is not allowed.
    - All motion must be generated by your ROS 2 node.

b. Pose source configuration
   To compute feedback control, your node must subscribe to a robot pose source. You will evaluate your controller using two different pose sources:
   - World (ground-truth) pose
   - Odometry (encoder-based) pose

   Your node must be configurable so that either pose source can be selected.

c. Experimental procedure
   Run the following two experiments:
   - Experiment 1 – World pose
     - Configure your node to use the world (ground truth) pose.
     - Run the simulation and observe path tracking performance.
   - Experiment 2 – Odometry pose
     - Configure your node to use encoder-based odometry.
     - Re-run the simulation using the same controller parameters.

d. Data analysis and discussion
   Using the data collected during both experiments, analyze and compare the robot's performance.
   <u>Based on this analysis, answer the following questions</u>:
   - Tracking accuracy.
     - How closely does the robot follow the square path in each case?
     - Are corner transitions smooth?
   - Motion behavior.
     - Does the robot rely on sideways motion to correct path errors?

- Pose source comparison.
    - How does tracking performance differ between world pose and odometry pose?
    - Explain the observed differences in terms of:
        - Feedback accuracy, and
        - Accumulated odometry error

<u>Submit your ROS 2 node source code, ensuring that it is clearly structured and properly commented, along with one screenshot per experiment showing the robot's behavior during a corner transition</u>.

Note 1: A reference Python file named *purePursuit_controller.py* is provided in the project repository package at the following path:
~/ros2_ws/src/yahboom_rosmaster/yahboom_rosmaster_projects/src/purePursuit_controller.py.
You may use this file as a starting template for your implementation.

Note 2: For a deeper understanding of the Pure Pursuit Algorithm, please refer to **Appendix I**.

2. Environmental mapping through SLAM.
In this task, you will use ROS 2 SLAM (Simultaneous Localization and Mapping) to build a 2D map of the simulated room while the robot autonomously follows the same square trajectory from Task 1.

During this task:
- SLAM estimates:
    - The robot's pose (localization), and
    - The map of the surrounding room (mapping).
- The robot must:
    - Use encoder-based odometry only
    - Be fully autonomous (no keyboard control)
    - Be controlled using purePursuit_controller.py

a. Motion and odometry requirements.
    - Configure the robot to use encoder-based odometry as the pose source.
    - Do not use world or ground-truth pose information.
    - The robot's motion must be fully autonomous (no keyboard control).

b. System architecture.
There are 3 independent components:

Component	What it does	How it is launched
SLAM	Builds map + estimates pose	Launch file
RViz	Visualizes mapping	Run manually
purePursuit_controller node	Moves robot	Run your node

You must:

1. Launch SLAM first
2. Open Rviz manually
3. Run your motion controller

c. Running SLAM.
Start the Gazebo simulation and launch the SLAM system using the provided launch file:

```
$: ros2 launch yahboom_rosmaster_projects gz_slam.launch.py
```

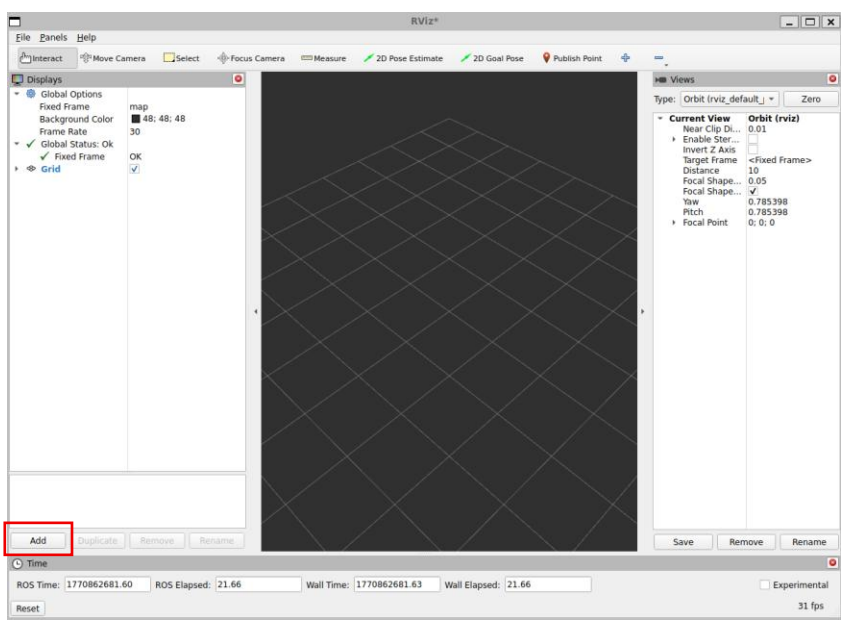
Note: This launch file only starts SLAM and the necessary simulation tools. It does not automatically configure RViz for you.

d. Visualization in RViz.

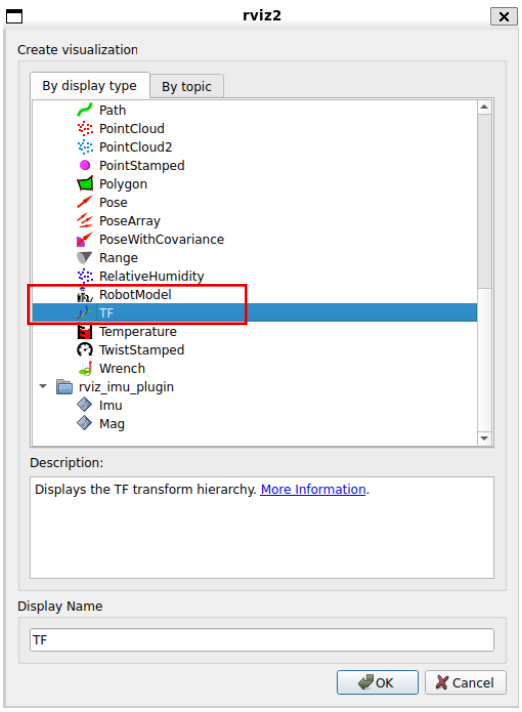
1. In a new terminal, run:

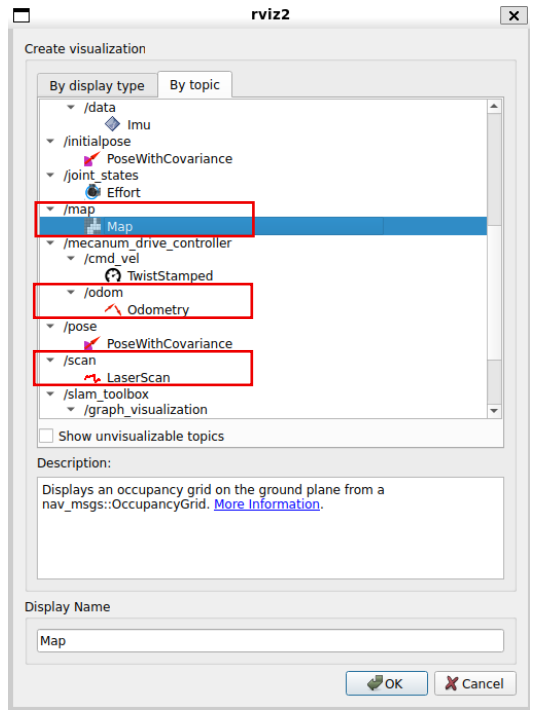
```
$: rviz2
```

2. Add required displays.
Click the "Add" button in the lower-left panel.



3. From the create visualization box, select the following:
   - By display type:
     - *TF* → shows coordinate frames and their relationships
     - *RobotModel* → shows the robot's 3D model
   - By topic:
     - *Map* → the 2D occupancy grid built by SLAM
     - *Odometry* → encoder-based motion estimate
     - *LaserScan* → the laser sensor readings

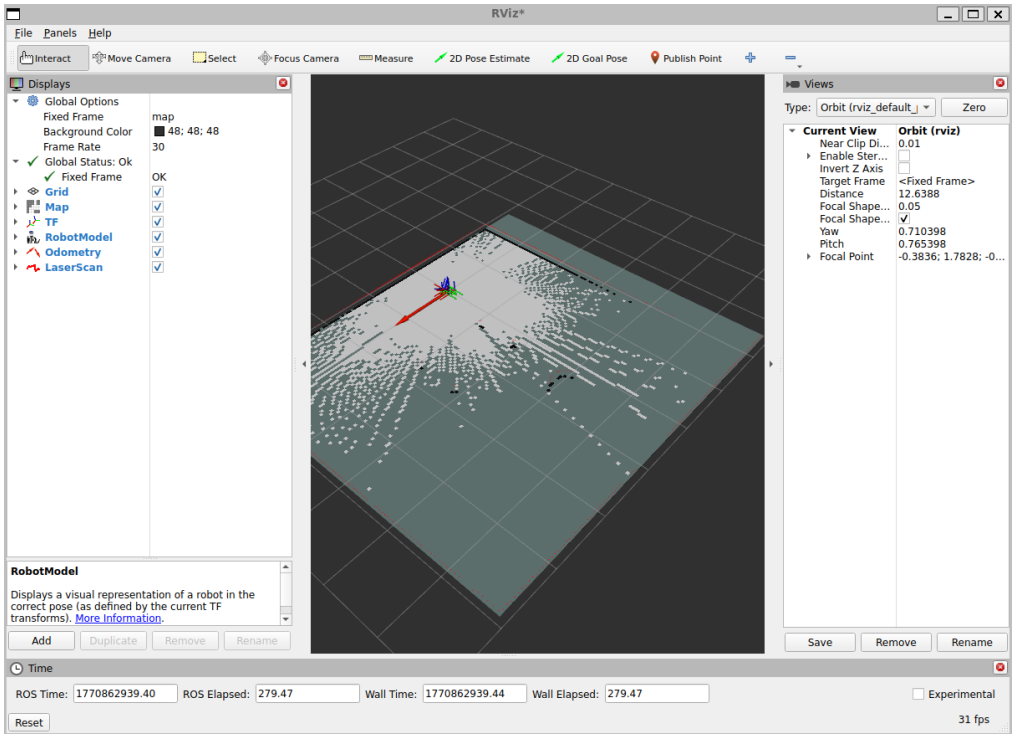




Click "OK" after adding each one.

At this stage:

- The robot model should be visible.
- The laser scan (usually red lines) should appear.
- The map may be mostly empty.
- The robot should not be moving.



e. Run your Pure Pursuit controller.
   Open a new terminal and run:
   $: ros2 run yahboom_rosmaster_projects purePursuit_controller.py
   What happens now?
   1. The controller commands the robot to follow the square path.
   2. Encoders produce odometry.
   3. SLAM receives:
      - Laser data
      - Odometry
   4. SLAM estimates:
      - The robot's corrected pose
      - The growing map

f. Data observation and discussion.
   During the SLAM process, observe and discuss the following:
   - Map quality.
     - Does the map room converge to a stable shape?
     - Do repeated passes over the same walls improve map consistency?
   - Localization improvement.

    - Compare the robot's estimated position with and without SLAM.
    - Does SLAM reduce drift caused by encoder-based odometry?
- Effect of motion control.
  As the robot completes multiple square cycles:
    - Does the map become more consistent?
    - Does localization become more stable?

<u>Submit screenshots from the RViz showing the room map</u>.

Appendix

I. Holonomic Pure Pursuit Algorithm (PPA)

*** What is Pure Pursuit?**

Pure Pursuit is a geometric path-following method. Instead of trying to "jump" directly to the next waypoint, the robot:

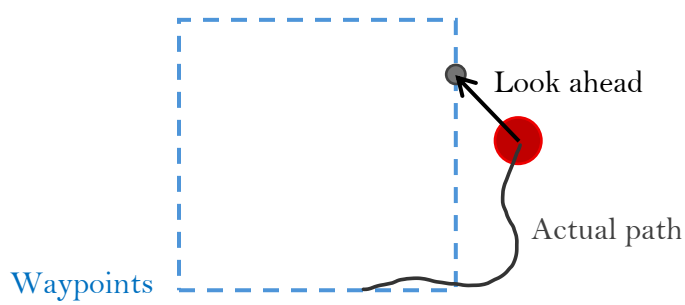
1. Finds where it is relative to the path
2. Looks ahead along the path by a fixed distance L_d
3. Selects a target point (called the lookahead point or "carrot")
4. Moves toward that point
5. Repeats this continuously

In that sense, the robot is chasing a carrot that is always a little bit ahead on the path. As the robot moves, the carrot moves too.

*** Visualizing the geometry**

Imagine:

- The square path is drawn on the floor
- The robot is slightly off the path
- A point exists on the path exactly L_d meters ahead of the robot



That point is the carrot. So, the robot does not aim for corners, it aims for that carrot.
A soon as it moves, the carrot moves forward too. This constant re-aiming is what produces smooth motion.

*** Why this produces smooth motion**

If the robot aimed directly at corners, it would drive straight, stop, rotate, continue, which would create sharp discontinuities.

Instead, Pure Pursuit continuously updates direction, never needs to stop, naturally smooth transitions. This smoothness comes from continuous geometric correction.

*** Standard vs Holonomic Pure Pursuit**

- **Standard (Non-Holonomic) case**
  For differential-drive robots:
  - They cannot move sideways
  - They can only move forward or rotate

  So standard Pure Pursuit computes:
  - A curvature
  - Forward velocity
  - Angular velocity

  This forces the robot to rotate when correcting lateral error.

- **Holonomic case (our mecanum robot)**
  The Rosmasterx3 can:
  - Move forward
  - Move sideways
  - Rotate

  Because it can move sideways, we do not need to rotate to correct path errors.
  Instead, we command:

$$v_x \rightarrow \text{forward velocity}$$
$$v_y \rightarrow \text{sideways velocity}$$
$$\omega = 0$$

  The robot corrects errors by sliding sideways.
  This is the key conceptual difference.

*** Main idea of the holonomic algorithm**

At every control cycle:

- **Step 1 – Find the closest point on the path**
  The algorithm projects the robot's position onto the square path.
  This answers: "Where am I relative to the path?"
  The result is $s_{closest}$, which is the distance along the path to the closest projection.
  <u>Why is this needed</u>? Because we need to measure progress along the path, not just distance in space.

- **Step 2 – Define the lookahead (carrot) point**
  We compute: $s_{goal} = s_{progress} + L_d$,
  Where L_d is the lookahead distance and $s_{progress}$ is how far along the path the robot has traveled.
  This ensures the carrot is always ahead of the robot.
  The lookahead ensures forward motion.

- **Step 3 – Compute error to carrot (world frame)**
  We compute: $dx = x_g - x$,
  $dy = y_g - y$.
  This is simply the vector from robot to carrot.
  It answers: "How far and in which direction is the carrot?".
  However, this vector is expressed in the world frame (the robot cannot use this directly).

- **Step 4 – Convert error to robot frame**
  The robot commands must be given in its own body frame. So, we rotate the error using the robot yaw: $x_r = \cos(\varphi)\, dx + \sin(\varphi)\, dy$,
  $y_r = -\sin(\varphi)\, dx + \cos(\varphi)\, dy$.
  Now:
  - x_r tells us how far ahead/behind the carrot is.
  - y_r tells us how far left/right the carrot is.

- **Step 5 – Generate velocity commands**
  We apply a simple proportional controller: $v_x = k_x \cdot x_r$,
  $v_y = k_y \cdot y_r$,
  Where k_x, k_y are gains.
  There is no angular velocity used.
  The robot corrects path errors by sliding sideways instead of rotating.

What happens at corners?
When the robot approaches a corner:

- The closest projection shifts to the next segment
- The carrot moves onto the new segment
- The error vector rotates naturally
- The robot gradually shifts direction

There is no special "corner logic": the geometry handles everything automatically.

Important parameters

- Lookahead distance L_d
  - Small → aggressive tracking, possible oscillation
  - Large → smoother but less accurate
- Gains k_x, k_y
  - Too small → slow correction
  - Too large → unstable motion
- Speed limits
  Protect the robot from unrealistic velocities.